CSCI 4100

Operating Systems and Architecture

Chapter 4: Concurrency and Threads

Topics

- 4.0 – Concurrency and Threads
- 4.1 – Thread Use Cases
- 4.2 – Thread Abstraction
- 4.3 – Simple Thread API
- 4.4 – Thread Data Structures
- 4.5 – Thread Life Cycle
- 4.6 – Implementing Kernel Threads
- 4.7 – Single-Threaded Processes
- 4.8 – Multi-Threaded Processes
- 4.9 – Alternative Abstractions

Concurrency

Concurrency refers to multiple activities that can happen at the same time.

- A modern OS performs many different tasks concurrently.
- Multiple hardware devices operate simultaneously.
- Multiple applications are running simultaneously.
- The OS must be responsive to the user while performing other tasks.
- Concurrency can be used to improve performance.

Application Concurrency

Concurrency is also used by applications.

- Network services must be able to handle multiple requests at a time.
- User interfaces must be responsive to the user while performing other tasks.
- Parallel programs must be able to run on multiple processors.
- Data management systems use concurrency to mask the latency of disk and network operations.

Threads

Concurrency is implemented using a set of interacting threads.

- A **thread** is a sequential stream of execution.
- Each thread behaves as if it has its own dedicated processor.
- Threads are scheduled, suspended, and resumed by the operating system so that only a subset of the threads are running at a given time.
- Coordination is required if multiple threads can access shared data structures.

Thread Use Cases

- Threads simplify the structure of programs that perform multiple independent activities.
- Threads improve responsiveness by having some threads execute background tasks while others quickly respond to user input.
- Threads improve performance by executing code on multiple processors at the same time.
- Threads improve performance by performing other tasks when one task is waiting for an I/O operation to complete.

Thread Abstraction

- The thread abstraction has two essential properties:
  1. Each thread represents a single execution sequence.
  2. Each thread is a separately schedulable task.
- A **thread scheduler** switches between running and non-running threads.
- Thread switches are transparent to the code running in each thread.
- This provides an abstraction where each thread is running on a processor with unpredictable speed.
- Thread programmers should not make any assumptions about how long it will take for operations to be performed.

Thread Programming

Why use this model for thread programming?

- It allows the application programmer to write code that is independent of the scheduler.
- It allows the kernel to schedule threads at will without worrying about program correctness.
- For independent threads, order of execution does not matter.
- For threads that share data, synchronization can be used to insure correctness.
- Even without scheduling, other outside factors can speed up or slow down program execution.

Simple Thread API

- A simple thread API might support the following operations:
  - `thread_create`: create a thread that executes a function with a given set of arguments.
  - `thread_yield`: suspend the calling thread so that another thread can run on the processor.
  - `thread_join`: wait for a thread to finish and return the exit value of the thread.
  - `thread_exit`: exit the current thread, storing the exit value in the thread's data structure and resume any thread waiting for it to finish.
- Together these operations implement threads as asynchronous procedure calls.

Fork-Join Parallelism

Applications can use these operations without additional synchronization to implement **fork-join parallelism**.

- A thread can use `thread_create` to create threads to perform work, and `thread_join` to wait for results.
- The parent can safely send data to a child by writing it before the child starts.
- The parent can safely receive data from a child by reading it after the child exits.
- The child threads work independently and do not share data with one another.

Thread Data Structures – The TCB

The OS uses a **thread control block (TCB)** to keep track of:

1. The state of the computation being performed by the thread.
   - A pointer to a stack that belongs only to that thread.
   - Copies of the processor registers used to suspend and resume the thread.
2. Metadata used to manage the thread.
   - The ID of the thread.
   - The scheduling priority of the thread.
   - The status of the thread: waiting for an event, ready to run, etc.

Thread Data Structures – Shared State

- Some state is shared between threads running in the same process.
- The code that the thread runs is shared by all threads.
- Global variables and heap allocated variables can be accessible by all threads.
- The OS does not provide any protection of per-thread data within a process.

Thread Life Cycle – Thread States

Over the course of its lifetime, a thread can be in the following states:

- INIT - the thread has been created and its per-thread data structures are being initialized.
- READY - the thread is available to run, but is not currently running.
- RUNNING - the thread is currently running on the processor.
- WAITING - the thread is waiting for an event to occur.
- FINISHED - the thread has completed its task.

Thread Life Cycle – State Change (1)

Threads can change states in the following ways:

- Once a new thread has been created and initialized, it is put on a ready list and its state changes from INIT to READY.
- When the thread scheduler chooses the next thread to run, it loads its register values into registers and changes its state from READY to RUNNING.
- When a thread yields a processor voluntarily the thread scheduler saves its registers and changes its state from RUNNING to READY.

Thread Life Cycle – State Change (2)

- When a running thread waits for an event to occur, the thread scheduler saves its registers, puts it on a waiting list, and changes its state from RUNNING to WAITING.
- When the event occurs, the thread scheduler changes the waiting thread's state from WAITING to READY.
- When a thread exits, the thread scheduler changes its state from RUNNING to FINISHED before freeing up its resources.

Implementing Kernel Threads

There are several different ways to implement threads:

- **Kernel threads** are implemented inside of the kernel, execute kernel code, and modify kernel data structures.
- **Single-threaded user processes** can invoke system calls whose handlers run concurrently with kernel threads.
- **Multi-threaded processes** can use multiple kernel threads to support multiple threads running in user mode.
- **User-level threads** use a user-level thread library and do not require any kernel support.

Creating a Thread

There are three steps to creating a thread:

1. Allocate per-thread state.
   - This includes the TCB and the stack.
2. Initialize per-thread state.
   - Set the new thread's registers in the TCB to contain correct initial values.
   - Start the thread in a stub function that calls the thread procedure, then calls `thread_exit`.
   - Push a frame on the stack for the stub along with an extra dummy frame whose return address is set to the stub (this will make sense later.)
3. Put the TCB on the ready list.

Deleting a Thread

There are two steps to deleting a thread:

1. Remove the thread from the ready list.
   - When the thread exits it moves to the FINISHED state.
   - The TCB for the thread is moved to a list of finished threads so it will not be run by the scheduler.
2. Free the per-thread state allocated for the thread
   - The thread can not safely de-allocate its own state as it needs a stack to operate.
   - Instead another thread is responsible for freeing its state.

Thread Context Switch

- A **thread context switch** suspends the currently running thread and resumes execution of another thread.
- To implement this we need to answer the following questions:
  - What triggers a context switch?
  - How does a voluntary context switch work?
  - How does an involuntary context switch differ from a voluntary one?
  - What thread should the scheduler choose to run next?
- The first three questions have to do with the **mechanism** used to switch threads.
- The last question has to do with thread scheduling **policy**.

Thread Context Switch Triggers

- A thread context switch can be triggered by a voluntary call into the thread library or an involuntary interrupt or processor exception.
- The `thread_yield`, `thread_join`, and `thread_exit` calls all suspend the calling thread and start running a new one.
- Interrupts and processor exceptions result in a call to an interrupt handler, which can switch threads or resume the running thread.
- A timer interrupt can be used to periodically switch threads.
- I/O event handlers may move waiting threads onto the ready list.
- When switching between threads, interrupts must be masked.

Voluntary Thread Context Switch (1)

The `thread_yield` procedure does the following:

1. Mask interrupts.
2. Get a TCB for another thread from the ready list.
3. If there are no threads to run, continue running the current thread.
4. Otherwise change state to READY, add the current thread to the ready list, and call `thread_switch` to switch to a new thread.
5. When a suspended thread resumes, its state is set to RUNNING
6. In either case, all threads on the finished list are de-allocated and interrupts are unmasked.

Voluntary Thread Context Switch (2)

The `thread_switch` procedure does the following:

1. Push the general register values onto the old thread's stack.
2. Save the old thread's stack pointer in its TCB.
3. Load the new thread's stack pointer into the SP register.
4. Restore the general register values from the new thread's stack.
5. Return to the calling procedure.
   - If the restored thread voluntarily suspended itself, this is `thread_yield`.
   - If the restored thread has not run yet, the processor jumps to the return address, which was set to the stub procedure.

Involuntary Thread Context Switch

An involuntary kernel thread context switch is handled like an interrupted user-level process:

1. The state of the thread is saved by the hardware and the handler.
2. The appropriate handler is run.
   - No mode switch is required since the kernel thread is running in kernel mode.
   - The kernel thread's stack can be used to save state and provide an execution context for the handler.
3. The state of the next ready thread is restored.

Single-Threaded Processes

- A single-threaded process is a thread with its own address space.
- A PCB includes all of the information in a TCB along with information about virtual memory mappings.
- The kernel's ready list can contain a mix of PCBs for processes and TCBs for kernel threads.
- When a user-level process switches to kernel mode it behaves exactly like a kernel thread.
- When a process is suspended, the PCB and kernel stack must store both kernel state and user-level state.
- This state is restored when a process resumes execution.

Multi-Threaded Processes – Kernel-Backed

Multi-threaded processes can be implemented using kernel threads.

- When a thread is created, the user library allocates a user-level stack, then performs a system call.
- The kernel allocates a TCB and interrupt stack, sets them up to start the user thread, and stores a pointer to the TCB in the PCB.
- After creating the thread, the kernel puts its TCB on the ready list and returns a unique thread identifier.
- Library calls for `join`, `yield`, and `exit` also call into the kernel to perform operations.

Multi-Threaded Processes – User-Level

Multi-threaded processes can be implemented without kernel support.

- User-level threads reduce dependencies on a specific OS and maximize portability.
- All thread data structures are implemented in the process.
- To the OS the process appears to be single-threaded.
- When an interrupt or trap occurs, all threads are suspended.
- A process can use upcalls to preempt user-level threads.

Multi-Threaded Processes – Hybrid Approach

Many systems use a hybrid of the kernel and user thread approaches.

- If `thread_join` is called after the targeted thread calls `thread_exit`, trapping into the kernel can be avoided.
- If each processor has a dedicated kernel thread, user-level threads can be multiplexed on top of the kernel threads.
- **Scheduler activations** can be used to provide additional support to user-level thread libraries.

Alternative Abstractions

There are a number of alternative ways of supporting concurrency.

- **Event-driven programming** uses asynchronous I/O operations to overlap high-latency I/O operations and processing.
- **Data parallel programming** splits a data set into multiple partitions and runs the same code in parallel to process the data.

In both cases the programmer can use a deterministic, sequential model to perform computations.

Event-Driven Programming

- **Asynchronous I/O** allows a single-threaded process to issue concurrent I/O requests without waiting for the results.
- At a later time, the OS can provide the response to the requests via signal handlers, event queues, or additional system calls.
- In **event-driven programming** a thread spins in a loop repeatedly processing the next I/O event.
- For each task, the thread maintains a **continuation** that stores the tasks current state and the next step it needs to take.

Data Parallel Programming

- **Data parallel programming** divides a data set among multiple processors.
- Each processor runs the same computation on its partition.
- The work on all partitions must complete before the next computation can be performed.
- Large data analysis tasks use systems like Hadoop to spread data across multiple servers.
- GPUs can be used for parallel multimedia processing.